```
class Nodo:
    def __init__(self, codigo, descripcion):
        self.codigo = codigo
        self.descripcion = descripcion
        self.siguiente = None



class ListaEnlazada:
    def __init__(self):
        self.cabeza = None

    def agregar(self, codigo, descripcion):
        nuevo = Nodo(codigo, descripcion)

        # Se agrega el nodo al final sin perder los anteriores
        if self.cabeza is None:
            self.cabeza = nuevo
        else:
            actual = self.cabeza
            while actual.siguiente is not None:
                actual = actual.siguiente
            actual.siguiente = nuevo

    def mostrar(self):
        actual = self.cabeza
        while actual is not None:
            print(actual.codigo, actual.descripcion)
```

```
            actual = actual.siguiente



class Pila:
    def __init__(self):
        self.items = []

    def apilar(self, elemento):
        # Se agrega el elemento a la cima
        self.items.append(elemento)

    def desapilar(self):
        # devuelve y elimina el último elemento
        # controla el caso de pila vacía
        if len(self.items) == 0:
            return None
        return self.items.pop()



class Cola:
    def __init__(self):
        self.items = []

    def encolar(self, elemento):
        # Se agrega el elemento al final
        self.items.append(elemento)

    def desencolar(self):
```

```
        # devuelve y elimina el primer elemento
        # controla el caso de cola vacía
        if len(self.items) == 0:
            return None
        return self.items.pop(0)

    # RETO 9: consultar el primer elemento sin retirarlo
    def consultar_primero(self):
        if len(self.items) == 0:
            return None
        return self.items[0]



# ==========================
# PRUEBAS: CASO 9 - IMPRENTA
# ==========================

print("=== LISTA DE TRABAJOS DE IMPRENTA ===")
lista = ListaEnlazada()
lista.agregar("I01", "Volantes")
lista.agregar("I02", "Tarjetas de presentación")
lista.agregar("I03", "Afiches")
lista.mostrar()

print("\n=== PILA DE TRABAJOS ===")
pila = Pila()
pila.apilar("Diseño de volantes")
pila.apilar("Diseño de tarjetas")
```

```
print("Trabajo desapilado:", pila.desapilar())
print("Trabajo desapilado:", pila.desapilar())
print("Pila vacía:", pila.desapilar())

print("\n=== COLA DE IMPRESIÓN ===")
cola = Cola()
cola.encolar("I01 - Volantes")
cola.encolar("I02 - Tarjetas")
cola.encolar("I03 - Afiches")

# Reto individual: consultar sin retirar
print("Primer trabajo pendiente:", cola.consultar_primero())

# Comprobar que sigue en la cola
print("Cola después de consultar:", cola.items)

# Retirar el primer trabajo
print("Trabajo atendido:", cola.desencolar())

# Consultar el nuevo primer trabajo
print("Siguiente trabajo pendiente:", cola.consultar_primero())

# Comprobar la cola vacía
cola_vacia = Cola()
print("Consulta de cola vacía:", cola_vacia.consultar_primero())
```

¿Qué se agregó para el caso 9?

El método nuevo es:

python

```
def consultar_primero(self):
    if len(self.items) == 0:
        return None
    return self.items[0]
```

- self.items[0] consulta el primer trabajo pendiente sin eliminarlo.
- self.items.pop(0) retira el primer trabajo de la cola.
- return None controla el caso en que la cola está vacía.

Importante: corregí _init_ a __init__, porque Python necesita los dos guiones bajos a cada lado para reconocer los constructores. Guarda el archivo como main.py y ejecuta las pruebas para comprobar que todo funciona.